Group X Progress Report:
My Group's Project Name

Aditya Tripuraneni, Vanessa Lai, Harmanpreet Singh Sagar
{tripuraa,laiv3,sagarh}@mcmaster.ca

1 Introduction

Distributed systems record their activity in logs, and engineers rely on these logs to find out when something has gone wrong. The volume of logsmakes manual inspection impractical: the HDFS cluster in our dataset produced over 11 million log lines in under two days, roughly 290,000 lines per hour [1, 2]. Failures are also rare, affecting only about 3% of sessions, so they are very hidden in the data. Supervised models can learn to recognize these failures, but they need labeled examples, and labeling logs at this scale takes time and money. This raises the question of whether labels are necessary at all. Our main research question is whether unsupervised models, which train without needing labels, can detect anomalies as well as supervised models. We will compare two supervised classifiers (logistic regression and XGBoost) with two unsupervised detectors (Isolation Forest and a DeepLog-style LSTM) using the same preprocessing and the same train/validation/test split.

2 Related Work

Here, talk about the related work you encountered for your approach. Cite at least 5 references. Refer to item 2. No one has done exactly your task? Write about the most similar thing you can find. This should be around 0.25-0.5 pages.

3 Dataset

3.1 Source and Collection

We use the HDFS_v1 dataset from Loghub [1, 3]. Its logs were generated by running benchmark workloads on a 203-node Hadoop Distributed File System (HDFS) cluster [1, 4] and contain 11,175,629 log lines, about 1.5 GB [2]. HDFS splits files into large blocks, replicates each block on multiple DataNodes (three by default), and records block locations on a NameNode [5]. In the data, every log line contains a block ID, and most logged block sizes are 64 MB (94% are 67,108,864 bytes). Grouping the lines by block ID yields one *session* per block [1, 2].

Sessions (block IDs)	575,061
Anomalous sessions	16,838 (2.93%)
Event types (of 30 templates)	29
Length: median / 99th pct / max	19 / 33 / 298

Table 1: Summary of the HDFS_v1 sessions. Counts agree with [2]; length statistics are our own computation.

3.2 Properties of the Data

One data point is one block session: the time-ordered sequence of log events for a single block, with a binary normal/anomalous label. Loghub defines 30 event templates (E1–E30), message types in which variable parts such as block IDs and IP addresses are replaced by wildcards [3]. Only 29 occur in the sessions, in agreement with [2]. Reading the template names, a typical normal session follows a regular lifecycle: a block is allocated (E22), received by three DataNodes (E5, E11, E9), recorded by the NameNode (E26) once per replica, then deleted (E23, E21), so the same events repeat three times, once per replica [5]. Anomalous sessions break this pattern. For example, `[E5, E5, E22, E7, E11, E10, E14, E7]` contains three exception events (E7, E10, E14) and never reaches E26.

Table 1 summarizes the data. It is highly imbalanced: 16,838 of the 575,061 sessions are anomalous [2], i.e., 2.93%, and we reproduced these counts from the Loghub files. Sessions are short (median 19 events), and length does not separate the classes (median 19 for normal, 20 for anomalous). We found no missing values and no duplicate block IDs.

3.3 Preprocessing

Our proposal planned to parse the raw logs into event templates with Drain [6]. We found that Loghub already provides parsed event templates and per-block event sequences, so we use those directly. The remaining conversion is handled by our own loader functions, which keeps the pipeline simple. The loader (1) joins each block's event sequence with its label by block ID, checking the join is exactly one-to-one; (2) converts each stored sequence string into a list of integer event IDs; (3) drops the `Type`, `Latency`, and `TimeInterval` columns, where `Type` holds the anomaly type that Loghub provides alongside the labels [1] and is populated for exactly the anomalous sessions, so using it would leak the label, and the other two fall outside our model input (the event sequence alone); and (4) encodes labels as 0 (normal) and 1 (anomalous).

3.4 Train, Validation, and Test Splits

Our dataset has a 70/10/20 train/validation/test split. We stratified the dataset on the binary label so each split keeps the same 2.93% anomaly rate. Without stratification, a random split could put too few of the rare anomalies into validation or test, and the four models would no longer be compared on the same class balance.

3.5 Annotation

We did not annotate any data. The labels were assigned by the dataset's authors, who manually labeled each block session as normal or anomalous using handcrafted rules [1]. We treat them as ground truth.

4 Features

Each block session is represented in one of two ways, depending on the model. In both cases, the model only uses the sequence of event IDs from the session (Section 3).

Event-count vectors. Logistic regression, XGBoost, and Isolation Forest use a count-based representation. For each session, we create a vector with one entry for every event type, where each value records how many times that event appears in the session. This is a common representation for HDFS session classification [2]. The event vocabulary is built using only the training split, resulting in 29 features. If an event type appears only in the validation or test set, it is ignored so that information from those splits does not leak into training.

Some events can occur many times in the same session, especially when a block is read repeatedly, which makes the raw counts highly skewed. To reduce this effect, we apply $\log(1+x)$ to each count before training the models. The main drawback of this representation is that it only captures how often events occur and completely ignores their order. This is one reason we also use an LSTM-based model.

Ordered event sequences. The DeepLog-style LSTM [7] uses the events in their original order. Each event type seen in the training data is assigned an integer token, along with an additional token for events that were never seen during training. The model is trained to predict the next event in a session. For each event, the previous 10 events are used as the input window and the current event is used as the target. At the beginning of a session, shorter windows are left-padded with a padding token.

We also add an end-of-session token to every sequence so that the model can learn when a normal session is expected to stop. This is important for unusually short sessions such as `[E22, E5]`, where the events themselves may look normal even though the session ends too early. Each token is mapped to a 16-dimensional embedding that is learned together with the LSTM, allowing the model to learn its own representation of the event types rather than relying on manually designed features.

Selection and augmentation. We do not perform feature selection or data augmentation. Since there are only 29 event types, all count-based features are kept. For logistic regression, the L2 regularization term helps limit the influence of features that are not useful for classification. As additional representations for the count-based models, we plan to test TF-IDF weighting and event n-gram counts. These may provide more information about which events are important while also preserving some local event ordering.

5 Implementation

Describe your model and implementation here. Refer to item 4. This may take around a page.

Isolation Forest. Isolation Forest [8] is an unsupervised method that flags sessions that are unusual

compared with the rest of the data. It builds random trees, each of which repeatedly splits the sessions on a random feature and a random value. Unusual sessions are isolated after only a few splits, while typical sessions need many, so a short average path length across the trees indicates an anomaly. We use the scikit-learn implementation with 100 trees, trained on the $\log(1+x)$ event-count vectors of the training split. Labels are used only afterwards, to choose a decision threshold on the validation split and to evaluate the model.

6 Results and Evaluation

Protocol. We use the same stratified 70/10/20 split with a fixed random seed for all models (Section 3). This gives us 402,541 training, 57,507 validation, and 115,013 test sessions. All results so far are based on this single split, which allows us to compare models on exactly the same test data. The validation set also contains 1,684 anomalous sessions, giving us enough examples to tune the models. We plan to add 5-fold cross-validation to see how much the results change across different splits (Section 7).

We use the validation set to choose hyperparameters and decision thresholds, then evaluate each model once on the test set. For the unsupervised models, labels are not used during training and are only used to select the decision threshold on validation.

Metrics. Since only 2.93% of sessions are anomalous, accuracy alone would be misleading. A model that predicts every session as normal would already achieve about 97.1% accuracy without detecting a single anomaly. Instead, we focus on precision, recall, and F1 for the anomalous class. We also report the area under the precision–recall curve (PR-AUC), calculated using average precision. This measures how well a model ranks anomalous sessions above normal ones across different thresholds and is more useful than ROC-AUC when the classes are highly imbalanced [9].

Each model produces a continuous anomaly score. We choose the threshold that gives the highest F1 on the validation set and use that same threshold for testing. For logistic regression, we also tested regularization strengths C from $\{10^{-2}, \ldots, 10^{3}\}$ and selected $C = 100$ based on validation PR-AUC. For the LSTM, we calculate a session's anomaly score using the largest $-\log p$ assigned to any of its actual next events. This means an event that the model considers highly unlikely can cause the session to be flagged as anomalous, similar to the probability cutoff used in DeepLog.

Model	P	R	F1	PR-AUC
Majority class	0.000	0.000	0.000	0.029
Supervised				
Logistic Regression	0.991	0.995	0.993	0.997
Unsupervised				
Isolation Forest	0.944	0.567	0.709	0.714
DeepLog LSTM	0.476	0.661	0.553	0.472
top-k rule (k=5)	0.478	0.131	0.205	0.107

Table 2: Test-set precision (P), recall (R), F1, and PR-AUC on the anomalous class. Thresholds were chosen on validation. XGBoost is still in progress.

Baseline. As a simple baseline, we use a majority-class predictor that labels every session as normal. Since it never predicts an anomaly, both its recall and F1 are zero. Its PR-AUC is also equal to the proportion of anomalous sessions in the dataset, which is 0.029.

Results. Table 2 summarizes our results so far. All models perform better than the majority-class baseline. We also found that the validation and test F1 and PR-AUC scores differ by less than 0.01 for each model, suggesting that the thresholds selected during validation work well on unseen data.

Logistic regression performs the best by a large margin, reaching an F1 of 0.993 and a PR-AUC of 0.997. This suggests that, when labels are available, simply knowing which events occur and how often they appear is enough to distinguish almost all normal and anomalous sessions. The model does not need to consider event order to achieve strong results on this dataset.

Among the unsupervised models, Isolation Forest achieves an F1 of 0.709. Its precision is high at 0.944, meaning that most sessions it flags are actually anomalous. However, its recall is only 0.567, so it misses almost half of the anomalies. One possible explanation is that some anomalous sessions have event counts similar to normal sessions, making them difficult to separate using count-based features alone.

The DeepLog LSTM has a higher recall of 0.661, meaning it detects more anomalies than Isolation Forest, but its precision drops to 0.476 because it also flags many normal sessions. Looking more closely at its mistakes, we found that many missed anomalies are short, incomplete sessions such as `[E22, E5]`. These patterns appear repeatedly in

the dataset, and since the unsupervised models are trained on all sessions, including the roughly 3% that are anomalous, the LSTM can learn to treat them as normal. On the other hand, many of its false positives are longer normal sessions where events from different replicas appear in an unusual order.

We also compared our probability-based anomaly score with the original DeepLog top-k approach, which flags a session when the actual next event is not among the model's top k predictions. With $k = 5$, this approach performs much worse, reaching an F1 of only 0.205 compared with 0.553 for our probability-based score. We found that some rare error events still appear among the top few predictions, even when their predicted probabilities are low. The top-k method therefore misses them, while the probability-based score can still identify them as unusual.

Adding the end-of-session token also made a noticeable difference. When we removed it, the LSTM's validation F1 dropped from 0.561 to 0.500, and its PR-AUC fell from 0.469 to 0.353. This suggests that learning when a session should end helps the model detect incomplete sequences that might otherwise look normal.

Overall, our results so far show a clear advantage for supervised learning on the HDFS dataset. Logistic regression performs particularly well using only event counts, while the unsupervised models struggle with anomalies that resemble normal sessions. Even the strongest unsupervised model, Isolation Forest, falls behind logistic regression by about 0.28 F1. This suggests that having labelled examples makes a substantial difference for anomaly detection in this dataset.

7 Feedback and Plans

We plan to use 5-fold cross-validation over the training and validation data, keeping the test set fixed, to measure how much each model's scores and chosen thresholds vary across folds.

7.1 TA Feedback

We received feedback from our TA, Hongyi, who suggested we include significance tests for our results. We plan to use McNemar's test to compare each unsupervised model against each supervised model. McNemar's test fits our setup because every model makes a binary prediction on the same test sessions, and it only looks at the sessions where the two models disagree. Because McNemar's test only tells us whether two models differ, we will also report 95% confidence intervals for F1 and PR-AUC to show how large the difference is.

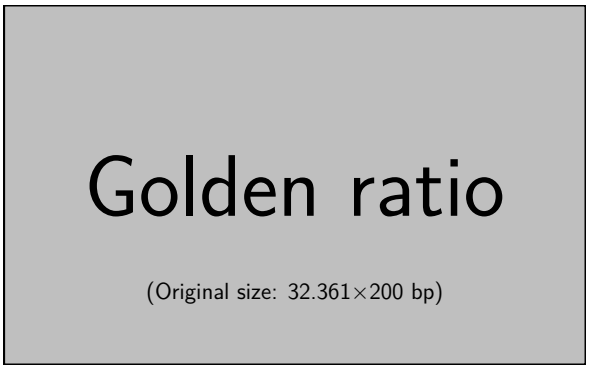


Figure 1: A figure with a caption that runs for more than one line. Example image is usually available through the `mwe` package without even mentioning it in the preamble.

7.2 Tables and figures

See Table 3 for an example of a table and its caption. See Figure 1 for an example of a figure and its caption.

7.3 Citations

Table 3 shows the syntax supported by the style files. We encourage you to use the natbib styles. You can use the command `\citet` (cite in text) to get "author (year)" citations, like this citation to a paper by **(author?)** [10]. You can use the command `\citep` (cite in parentheses) to get "(author, year)" citations [10]. You can use the command `\citealp` (alternative cite without parentheses) to get "author, year" citations, which is useful for using citations within parentheses (e.g. 10).

7.4 References

Many websites where you can find academic papers also allow you to export a bib file for citation or bib formatted entry. Copy this into the `custom.bib` and you will be able to cite the paper in the LaTeX. You can remove the example entries.

7.5 Equations

An example equation is shown below:

$$A = \pi r^2 \tag{1}$$

Labels for equation numbers, sections, subsections, figures and tables are all defined with the `\label{label}` command and cross references to them are made with the `\ref{label}` command.

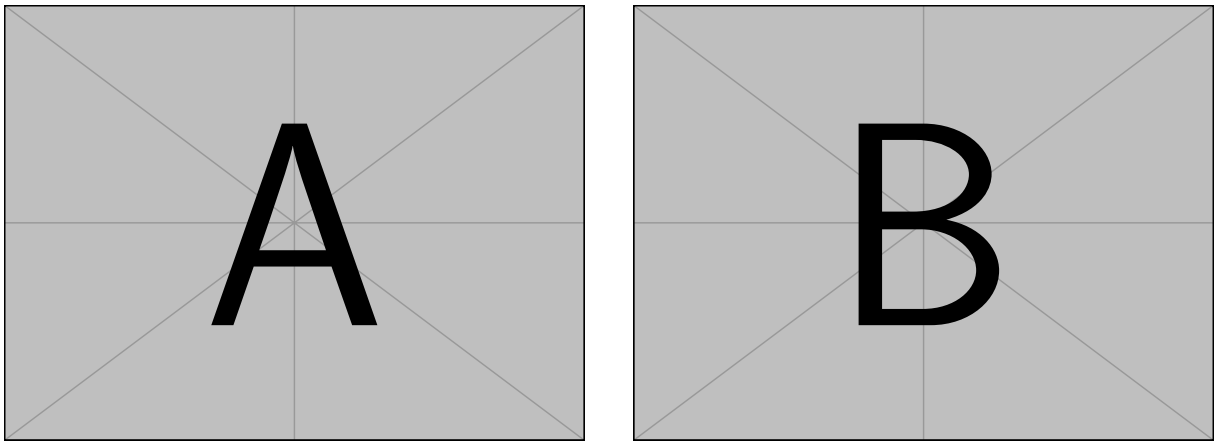


Figure 2: A minimal working example to demonstrate how to place two images side-by-side.

Output	natbib command	ACL only command
[10]	`\citep`	
10	`\citealp`	
(author?) [10]	`\citet`	
[**(year?)**]	`\citeyearpar`	
(author?)'s (**(year?)**)		`\citeposs`

Table 3: Citation commands supported by the style file.

This an example cross-reference to Equation 1. You can also write equations inline, like this: $A = \pi r^2$.

Team Contributions

Write in this section a few sentences describing the contributions of each team member. What did each member work on? Refer to item 7.

References

[1] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," arXiv:2008.06448, 2020.

[2] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in *Proc. IEEE 27th Int. Symp. Software Reliability Engineering (ISSRE)*, 2016, pp. 207–218.

[3] LogPAI, "Loghub: System log datasets," https://github.com/logpai/loghub, accessed: Nov. 2, 2026.

[4] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proc. 22nd ACM Symp. Operating Systems Principles (SOSP)*, 2009, pp. 117–132.

[5] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop distributed file system," in *Proc. IEEE 26th Symp. Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1–10.

[6] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2017, pp. 33–40.

[7] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017, pp. 1285–1298.

[8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 Eighth IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.

[9] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, p. e0118432, 2015.

[10] D. Gusfield, *Algorithms on Strings, Trees and Sequences*. Cambridge, UK: Cambridge University Press, 1997.

[11] R. K. Ando and T. Zhang, "A framework for learning predictive structures from multiple tasks and unlabeled data," *Journal of Machine Learning Research*, vol. 6, pp. 1817–1853, Dec. 2005.

[12] G. Andrew and J. Gao, "Scalable training of L1-regularized log-linear models," in *Proceedings of the 24th International Conference on Machine Learning*, 2007, pp. 33–40.

[13] M. S. Rasooli and J. R. Tetreault, "Yara parser: A fast and accurate dependency parser," *Computing*

Research Repository, vol. arXiv:1503.06733, 2015, version 2. [Online]. Available: http://arxiv.org/abs/1503.06733